UNIVERSIDAD AUSTRAL

INGENIERÍA

Procesamiento de Imágenes

UA – LCD – Año 2026

Trabajo Práctico N.° 2 — Informe

Di Rienzo Bernardo, Miles Benjamín, Sanfilippo Lautaro, Ziadi Marcos

Problema 1 — Detección y clasificación de monedas y dados

Enunciado

Se dispone de la imagen monedas.jpg, que consiste en un fondo de intensidad no uniforme sobre el cual se hallan dados y monedas de distinto valor y tamaño. Se debe elaborar un algoritmo capaz de procesar dicha imagen y resolver tres puntos: (A) segmentar de manera automática las monedas y los dados; (B) clasificar los distintos tipos de monedas y realizar un conteo automático; y (C) determinar el número que representa la cara superior de cada dado y realizar un conteo automático.

En cada punto, el script elaborado debe informar y mostrar los resultados en cada una de las etapas de procesamiento.

Punto A | Segmentar las monedas y los dados de manera automática

¿Qué distingue a las monedas y a los dados del fondo? Tres propiedades que usamos:

1. **Color.** El fondo es gris, de modo que su croma es casi nulo aunque su brillo cambie de izquierda a derecha. Las monedas doradas y bimetálicas sí tienen color.
2. **Brillo.** Las monedas plateadas y los dados casi no tienen color, pero son mucho más claros que el fondo.
3. **Forma y tamaño.** Las monedas son círculos de tres tamaños bien separados entre sí. Los dados son cuadrados y por eso se alejan del círculo.

Idea: pasar a un espacio de color que separe el brillo del color, binarizar con los dos criterios y limpiar la máscara con operaciones morfológicas.

Para ello, llevamos a cabo el siguiente procedimiento:

1. **Suavizado gaussiano**: Se aplica un filtro gaussiano con desvío $\sigma = 2$ para reducir el ruido fino de la textura del fondo antes de binarizar.
2. **Pasaje a CIELAB**: Se obtienen el canal L (brillo) y el croma, calculado como la distancia de los canales a y b al gris neutro: croma = $\sqrt{((a - 128)^2 + (b - 128)^2)}$. Como el fondo es gris, su croma es casi cero aunque cambie su brillo: el croma no depende del degradé de iluminación.

Figura 1 — Imagen original, canal L (brillo) y croma (color).

3. **Binarización**: Un píxel pertenece a un objeto si su croma es mayor a 4 (monedas doradas y bimetálicas) o si su brillo L supera un umbral (monedas plateadas y dados). El umbral de brillo lo calcula el método de Otsu usando solo los píxeles claros (L > 150), porque el fondo, que se aclara hacia la derecha, se mezclaría con los objetos; el valor obtenido es 194. La máscara es la unión (OR) de ambos criterios.

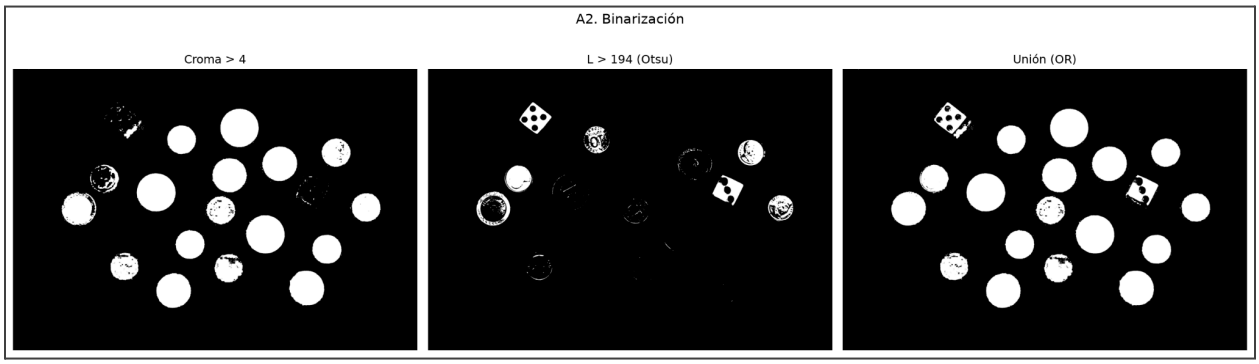

Figura 2 — Binarización por croma, por brillo (umbral de Otsu) y unión de ambas.

4. **Operaciones morfológicas**: Se aplica una clausura con elemento elíptico de 31×31, que tapa las grietas de la máscara. Luego se rellenan los huecos con cv2.floodFill: se inunda el fondo desde la esquina (0, 0) y lo que no se inunda es objeto; así se tapan los puntos oscuros de los dados. Por último, una apertura elíptica de 61×61 elimina el ruido y deja la máscara final.

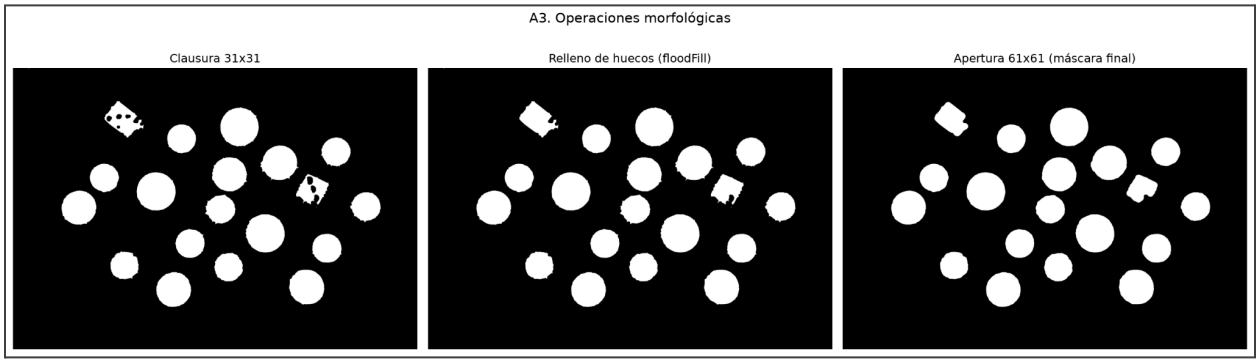

Figura 3 — Clausura, relleno de huecos y apertura (máscara final).

5. **Componentes conexas**: Con cv2.connectedComponentsWithStats se etiqueta cada isla blanca de la máscara: cada una es una moneda o un dado.

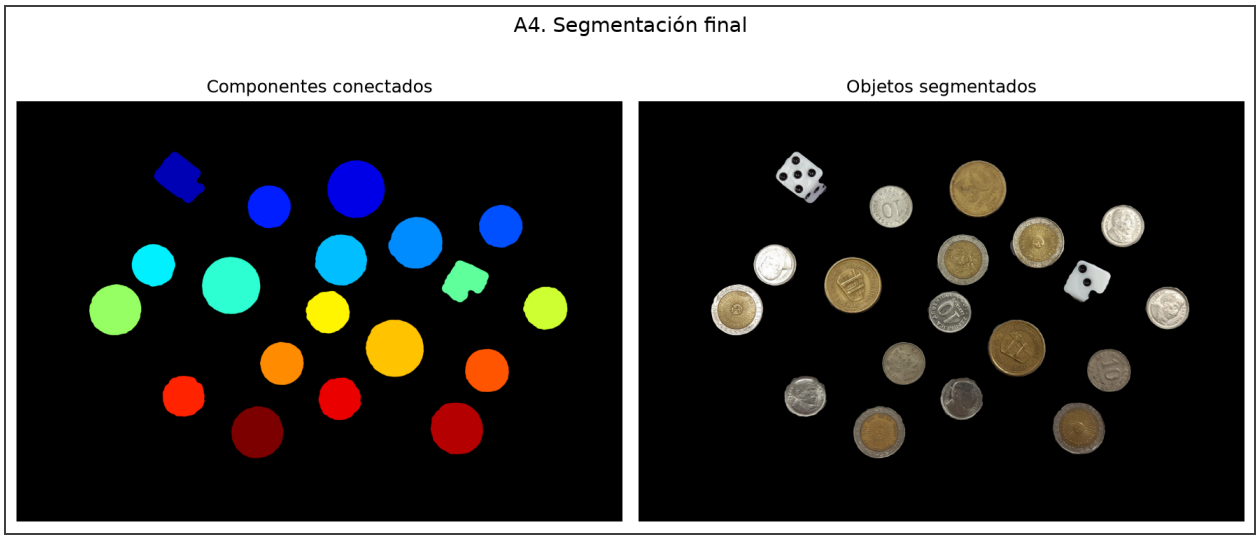

Figura 4 — Componentes conexas y objetos segmentados.

Resultado de la etapa: se detectan 19 objetos: 17 monedas y 2 dados.

Punto B | Clasificar los tipos de monedas y realizar un conteo automático

Idea: usar dos criterios en cascada. La forma separa las monedas de los dados y el área separa los tipos de moneda.

Para ello, llevamos a cabo el siguiente procedimiento:

1. **Medidas de cada objeto**: Para cada componente se obtiene su contorno con cv2.findContours, su área con cv2.contourArea y su perímetro con cv2.arcLength.
2. **Factor de forma**: Se calcula $\rho = 4 \cdot \pi \cdot \text{Área} / \text{Perímetro}^2$, que vale 1 para un círculo perfecto y baja en figuras con esquinas. Si $\rho > 0{,}8$ el objeto es una moneda: las monedas dan valores entre 0,86 y 0,89. Si no, es un dado: los dos dados dan 0,691 y 0,708.
3. **Clasificación por área**: Las monedas tienen tres tamaños bien separados, de modo que se agrupan sus áreas con k-means de 3 grupos (cv2.kmeans): el grupo de menor área corresponde a 10 centavos, el del medio a 1 peso y el de mayor área a 50 centavos. Los centros de los grupos son 58 090, 85 496 y 105 682 px. Las áreas medidas son: 10 centavos entre 55 108 y 60 628 px, 1 peso entre 83 826 y 87 067 px y 50 centavos entre 105 298 y 106 104 px. Así no hace falta fijar umbrales a mano.

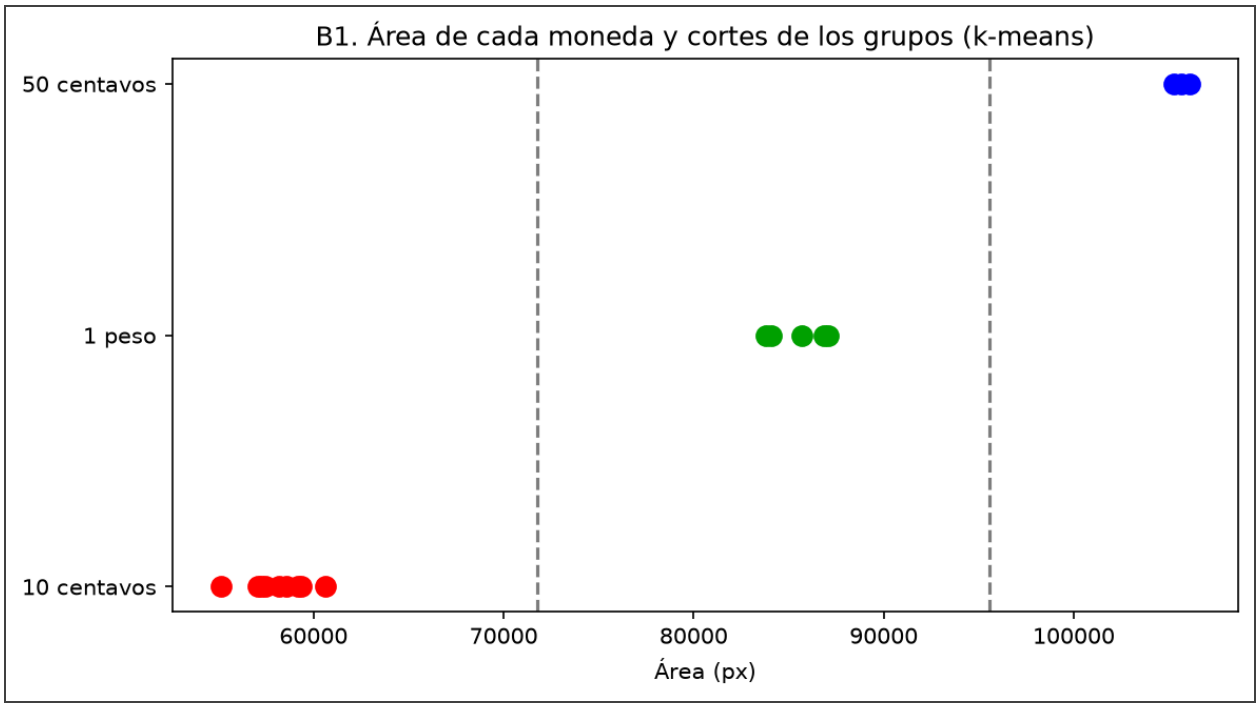

Figura 5 — Área de cada moneda y cortes entre los grupos de k-means.

4. **Conteo**: Se cuenta cuántas monedas hay de cada tipo y se suma su valor.

Figura 6 — Monedas clasificadas por área.

Tipo de moneda	Cantidad	Valor unitario	Subtotal
10 centavos	9	$0,10	$0,90
1 peso	5	$1,00	$5,00
50 centavos	3	$0,50	$1,50
Total	**17**		**$7,40**

Punto C | Determinar el valor de la cara superior de cada dado y realizar un conteo automático

Idea: los puntos (pips) de la cara superior se ven grandes y redondos, mientras que los de las caras laterales se ven chicos y estirados por la perspectiva. El valor del dado es la cantidad de pips de la cara superior.

Para ello, llevamos a cabo el siguiente procedimiento sobre cada dado:

1. **Silueta del dado**: Se rellena la silueta con el casco convexo del contorno (cv2.convexHull), porque la máscara de la segmentación puede tener muescas donde hay un punto oscuro en el borde.
2. **Píxeles oscuros**: Sobre la imagen en escala de grises con filtro de mediana de 5×5, se binariza con el umbral de Otsu calculado solo con los píxeles del dado (117 y 116 para los dos dados). Luego una apertura elíptica de 5×5 limpia los puntos sueltos y una clausura elíptica de 15×15 tapa los reflejos y las mordidas dentro de cada pip.
3. **Candidatos a pip**: Se obtienen las componentes conexas de los píxeles oscuros y para cada una se calculan su área y su factor de forma ρ.
4. **Filtro por tamaño y forma**: Un candidato es un pip de la cara superior si es redondo ($\rho > 0{,}65$) y su área es al menos la mitad de la del pip redondo más grande de ese dado; así el criterio se adapta al tamaño de la foto. Los pips de la cara superior miden entre 1 586 y 2 216 px, con ρ entre 0,78 y 0,90. Los descartados, que corresponden a las caras laterales, miden entre 683 y 1 072 px, con ρ entre 0,26 y 0,59.

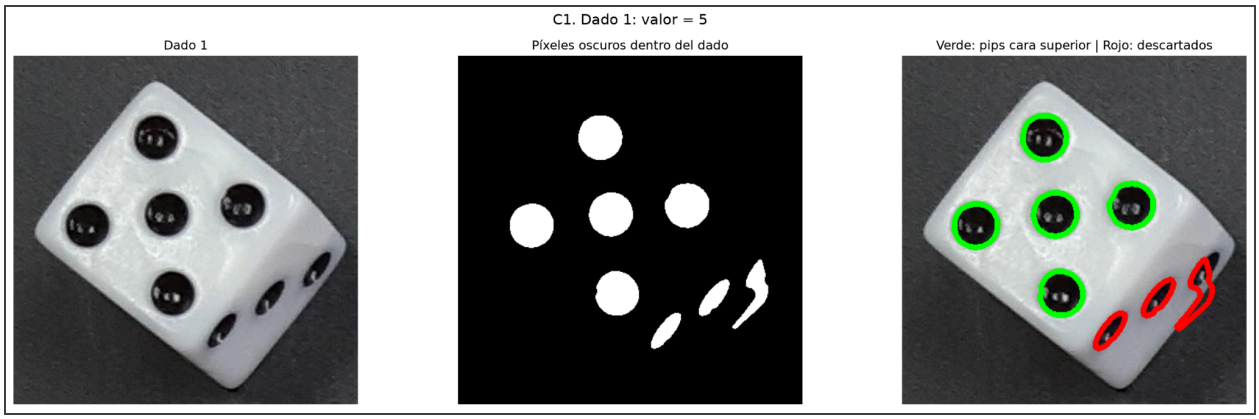

Figura 7 — Dado 1: recorte, píxeles oscuros y pips detectados (verde: cara superior, rojo: descartados).

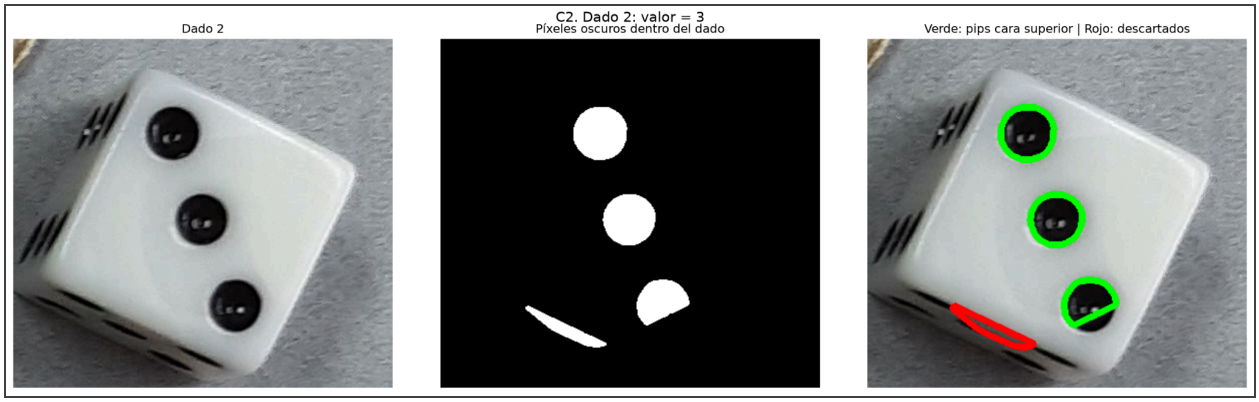

Figura 8 — Dado 2: recorte, píxeles oscuros y pips detectados (verde: cara superior, rojo: descartados).

5. **Valor del dado**: Es la cantidad de pips de la cara superior: el dado 1 tiene 5 y el dado 2 tiene 3, con una suma de 8.

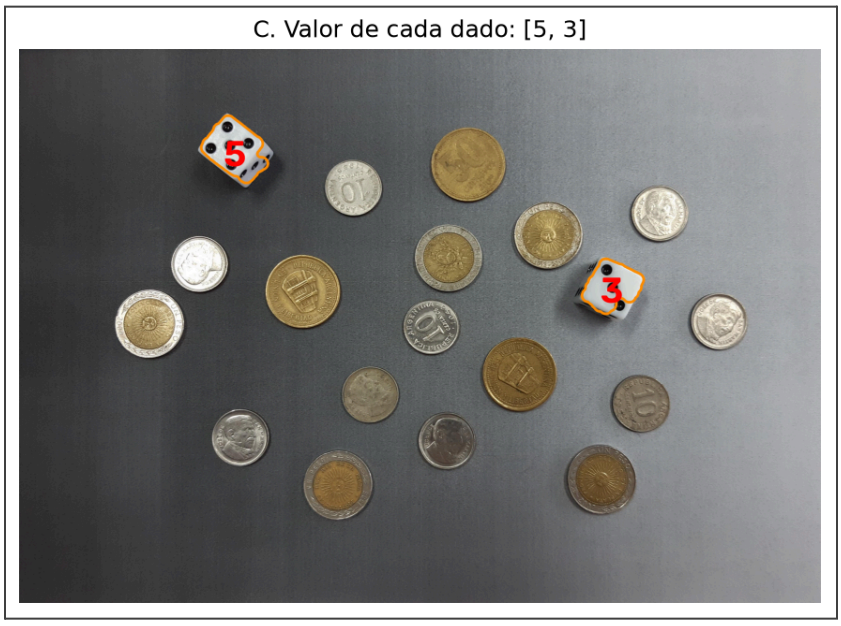

Figura 9 — Valor de la cara superior de cada dado.

Resultados

Resultado final sobre la imagen monedas.jpg:

Figura 10 — Monedas clasificadas y valor de los dados.

- Se detectan 19 objetos: 17 monedas y 2 dados.
- Monedas: 9 de 10 centavos, 5 de 1 peso y 3 de 50 centavos, con un valor total de $7,40.
- Dados: valores 5 y 3, con una suma de 8.

Problema 2 — Detección de patentes

Enunciado

En la Figura 2 se muestra una de las doce imágenes (archivos img<id>.png), las cuales representan la vista anterior o posterior de diversos vehículos. En cada una de ellas se puede visualizar las correspondientes patentes.

Se debe elaborar un algoritmo capaz de procesar cada una de estas imágenes y resolver los siguientes puntos en cada uno de ellas:

A. Detectar automáticamente la placa patente y segmentar la misma.
B. Implementar un algoritmo de procesamiento que segmente los caracteres de la placa patente detectada en el punto anterior.

Punto A | Detectar automáticamente la placa patente y segmentar la misma

Qué distingue a una patente del resto de la foto? Tres propiedades que usamos:

1. **Estructura de bordes verticales**. Los caracteres claros sobre fondo oscuro generan, al recorrer una fila de izquierda a derecha, múltiples transiciones "oscuro → claro". Pocas regiones de una escena vehicular presentan una densidad local alta de ese tipo de borde.
2. **Consistencia de tamaño**. Los 6 caracteres tienen bounding boxes de dimensiones casi idénticas, lo que permite distinguirlos del ruido y del marco por descriptores geométricos simples (área, altura, relación de aspecto).
3. **Regularidad espacial**. Los caracteres están alineados sobre una recta y equiespaciados, con un intervalo entre letras y dígitos. Si un caracter no se detecta, su posición puede inferirse de los demás.

Etapa 1 | Encontrar la patente en la foto

Idea: estimar la densidad local de bordes verticales oscuro→claro y seleccionar la ventana donde es máxima.

Para ello, llevamos a cabo el siguiente procedimiento:

1. **Recorte de márgenes**: Se excluyen el 20% izquierdo, el 20% derecho y el 20% superior de la imagen, donde por la geometría de la escena no aparecen patentes. Reduce el espacio de búsqueda y los falsos positivos.

Figura 1 — Recorte de márgenes.

2. **Máscara de píxeles oscuros**: Conversión a espacio de color HSV y umbralización del canal de valor (V): V < 80. Se trabaja solo con luminosidad, sin considerar tono ni saturación.

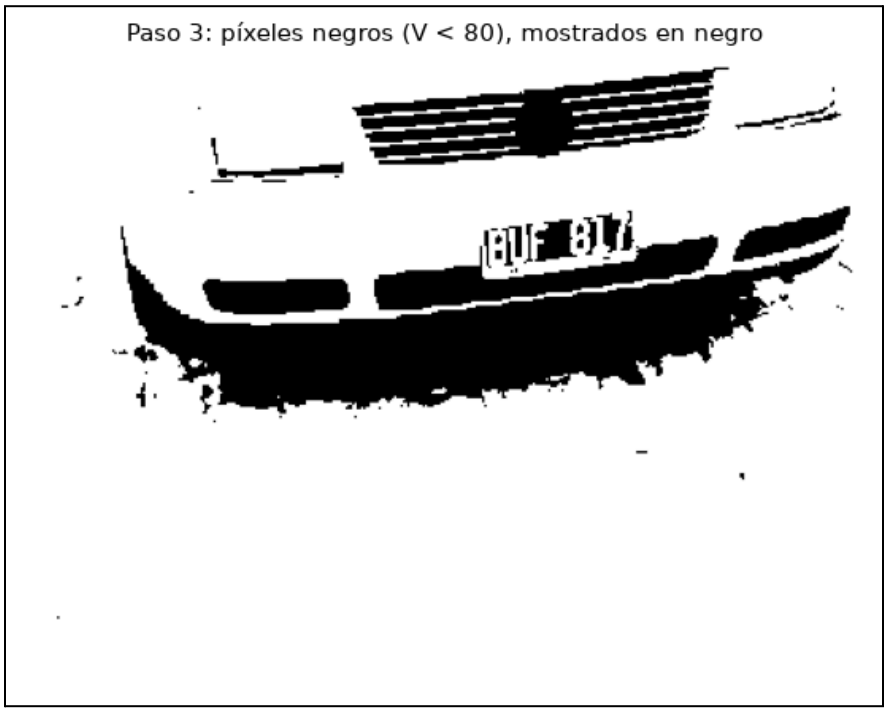

Figura 2 — Máscara de píxeles oscuros.

3. **Mapa de transiciones**: Para cada pixel (x, y), se marca 1 si el píxel anterior en la fila es oscuro y el actual no. Equivale a una primera diferencia horizontal [1, -1] aplicada a la máscara binaria, con rectificación (solo el signo positivo): detecta los bordes izquierdos de los trazos claros.

Figura 3 — Mapa de transiciones.

4. **Erosión con elemento estructurante vertical** (3x1): Un píxel sobrevive solo si tiene vecinos activos arriba y abajo. Los bordes de los caracteres son segmentos verticales de varios píxeles; el ruido (tipo sal y pimienta) son píxeles aislados que se eliminan.

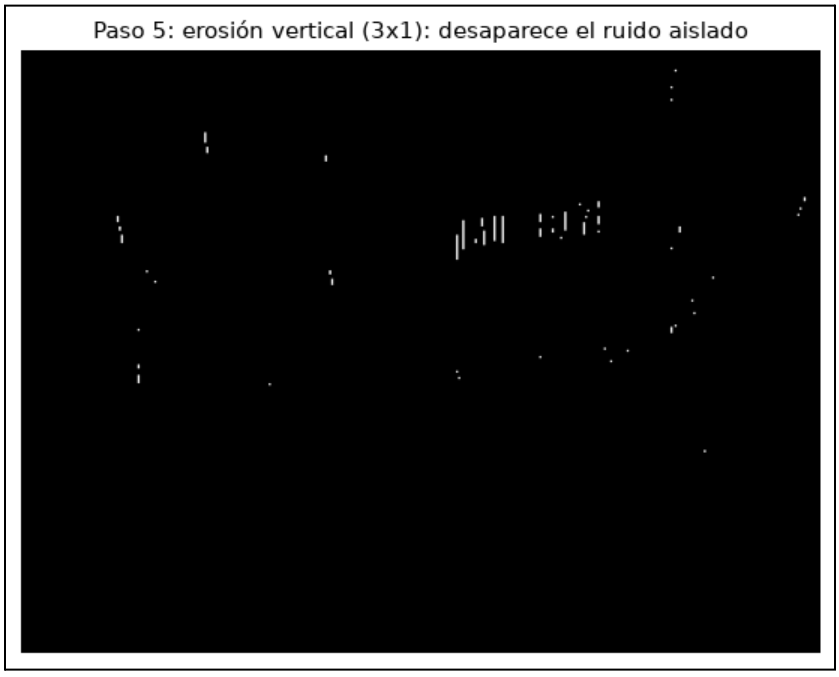

Figura 4 — Erosión.

5. **Dilatación con el mismo elemento** (3x1): Restituye el espesor de los trazos que sobrevivieron (Apertura).

Figura 5 — Dilatación.

6. **Filtro de promedio**: Convolución con un núcleo constante normalizado, de dimensiones similares a una patente (11% x 9% de la imagen). La respuesta en cada posición es la fracción de píxeles de borde en esa ventana, es decir, la densidad local de bordes.

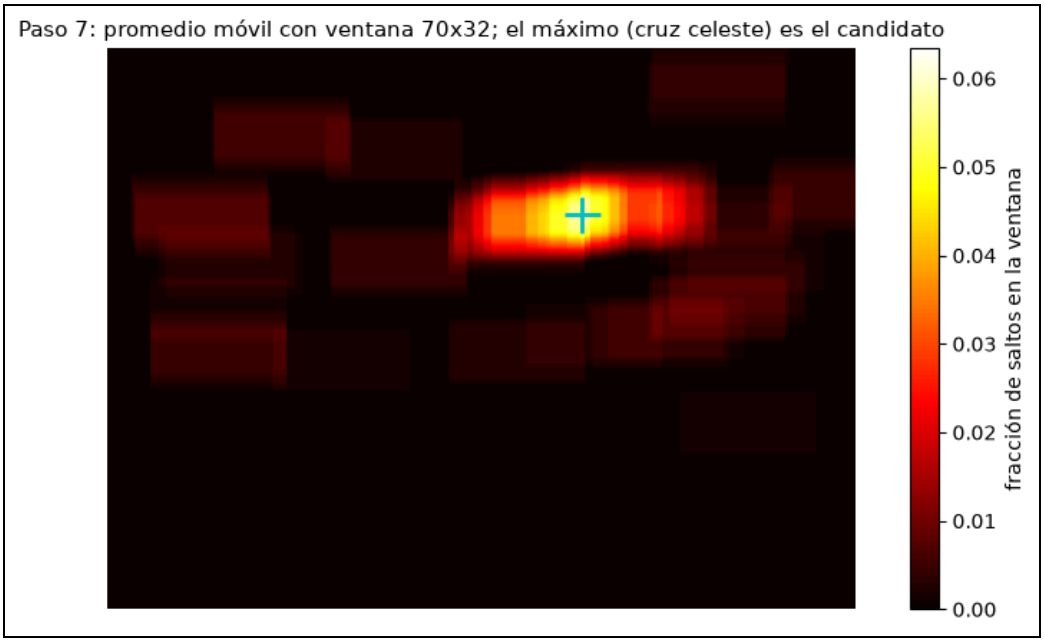

Figura 6 — Filtro de promedio.

Máximo global: La posición de máxima respuesta es la candidata. Para que no se generen falsos máximos, se usó una extensión de borde constante (cero).

7. **Refinamiento por centroide**: Se calcula el centroide de los píxeles de borde en una vecindad de 2W (22%) x 2H (18%) alrededor del máximo. De esta forma, se acerca la ubicación del punto al centro del texto aunque la ventana ganadora haya quedado un poco desplazada.

Figura 7 — Refinamiento por centroide.

8. **Extracción de la región de interés**: Se recorta un rectángulo de 16% x 14% de la imagen centrado en el punto. Si el punto está cerca del borde, la ventana se desplaza en lugar de truncarse, para que todos los recortes tengan igual tamaño.

Figura 8 — Extracción de la ROI.

Resultado de la etapa: el area recortada contiene dentro la patente en 12 de 12 imágenes.

Paso 9: zona que se va a recortar (verde)

Paso 9: zona que se va a recortar (verde)

Paso 9: zona que se va a recortar (verde)

Paso 9: zona que se va a recortar (verde)

Paso 9: zona que se va a recortar (verde)

Paso 9: zona que se va a recortar (verde)

Paso 9: zona que se va a recortar (verde)

Paso 9: zona que se va a recortar (verde)

Figura 9 — Todas las patentes detectadas.

Etapa 2 | Segmentación del recorte

Idea: convertir la ROI en una imagen binaria donde cada caracter sea una región conexa aislada del fondo, y extraer esas regiones.

Para ello, llevamos a cabo el siguiente procedimiento:

1. **Conversión a escala de grises**: Reduce la imágen a un único canal de intensidad

Figura 10 — Conversión a escala de grises.

2. **Estiramiento lineal de contraste** (Normalización min-max): Transformación afín que lleva la intensidad mínima a 0 y la máxima a 255, de modo que un umbral global fijo sea válido independientemente de la exposición de la imágen.

Figura 11 — Normalización Min-Max.

3. **Umbralización global fija** (T=127): I > 127 → 255, de lo contrario 0. Los caracteres quedan en primer plano (blanco) sobre el fondo oscuro de la patente,

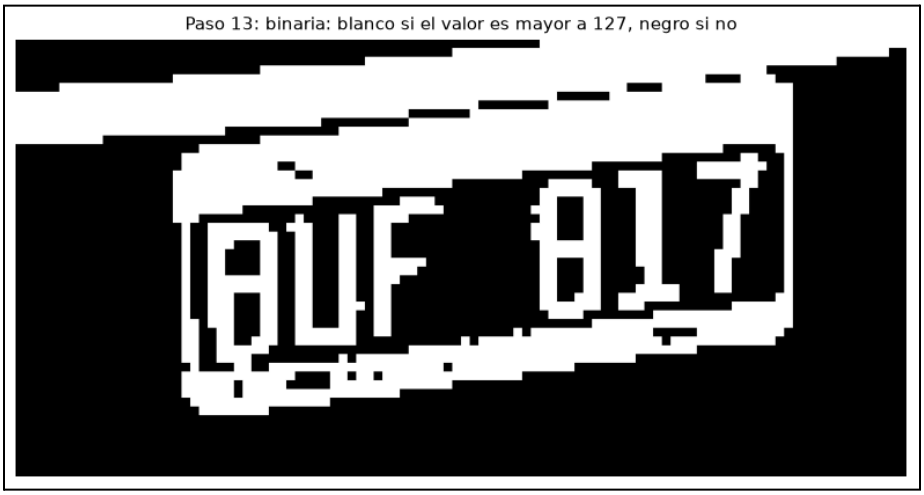

Figura 12 — Umbralización global fija.

4. **Etiquetado de componentes conexas**: cv2.connectedComponentsWithStats con vecindad=4; entrega para cada componente su bounding box y su área. La vecindad=4 evita unir regiones que solo se tocan en diagonal.

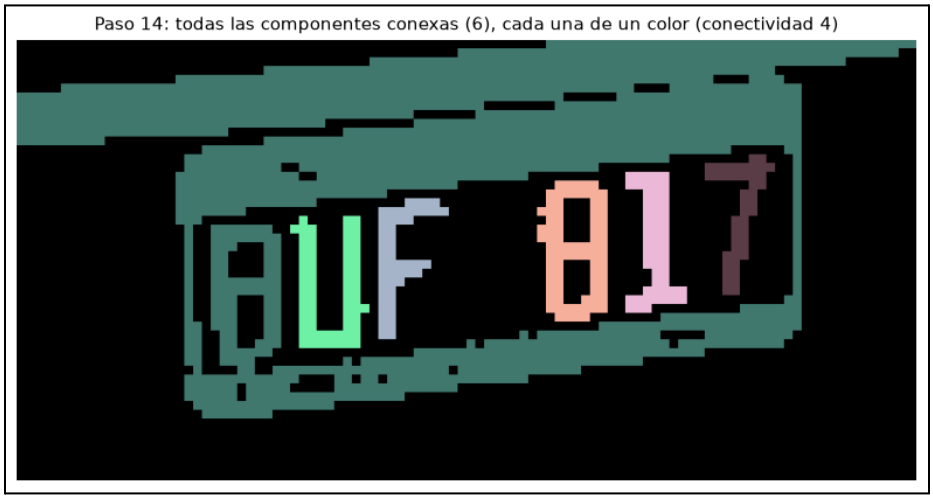

Figura 13 — Etiquetado de componentes conexas.

5. **Filtrado por descriptores geométricos**: Se descartan las componentes con áreas < 6px (ruido), altura menor al 20% o mayor al 70% de la altura de la ROI, o con relación de aspecto inválida (más anchas que altas).

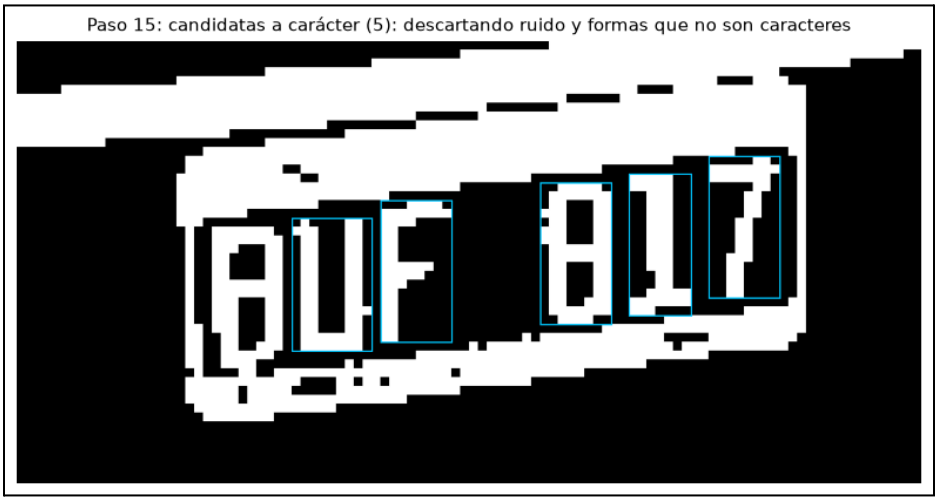

Figura 14 — Filtrado por descriptores geométricos.

Etapa 3 | Selección, normalización e inferencia geométrica

Selección por consistencia de área

Los bounding boxes de los 6 caracteres tienen áreas casi iguales; el ruido y el marco no. El procedimiento es:

1. Se calcula el área de cada bounding box (ancho x alto).
2. Cada caja “vota” por las que tienen un área dentro de una tolerancia relativa del ±25% de la suya.
3. Se adopta el área con más votos (consenso) y se conserva el grupo de cajas compatibles. Si hay más de 6, se retienen las 6 más próximas al área de consenso.

Cómo funciona la votación?

1. Calcular el área de cada caja: ancho x alto, no la cantidad de píxeles blancos.
2. Votar: cada caja recorre todas las demás, incluida ella misma, y cuenta cuántas tienen un área “parecida” a la suya. Parecida significa que la diferencia es de como máximo el 25% de su propia área.
3. Elegir la referencia: gana el área con más votos. Si hay empate, gana la primera de la lista.
4. Armar el grupo: se conservan todas las cajas dentro de ±25% del área de referencia.
5. Recortar a 6: si el grupo tiene más de 6 cajas, quedan las 6 más cercanas a la referencia.

La idea es que los caracteres reales se voten entre sí, porque son muchos y parecidos. El ruido queda aislado con pocos votos.

Figura 15 — Cajas que pasaron luego de la votación.

Normalización del tamaño de los bounding boxes

Algunas cajas quedan truncadas (no abarcan todo el caracter). Como los caracteres tienen dimensiones iguales, se toma la caja de mayor área y se reconstruye cada caja con ese ancho y alto, centrada en el centro de la caja original. Cabe aclarar que se utiliza el centro del bounding box y no el centroide (centro de masa) del caracter. Todo el procesamiento posterior usa estas cajas normalizadas.

Figura 16 — Normalización del tamaño de los bounding boxes

Inferencia de caracteres faltantes

La inferencia explota la alineación y la equidistancia de los caracteres:

1. **Estimación de la inclinación**. Se traza la recta que une los centros de la primera y la última caja; su pendiente da el ángulo de inclinación de la patente en el plano de la imágen. Este ángulo se reutiliza en la Etapa 4.

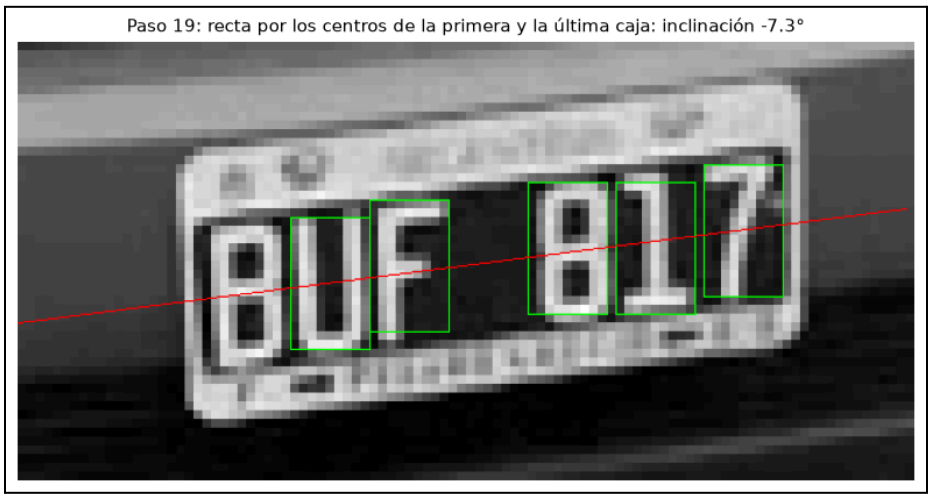

Figura 17 — Trazado de linea que une los centroides de los simbolos.

2. **Banda de referencia**. Con la misma pendiente, se trazan dos rectas paralelas por la caja de referencia: una por su vértice superior izquierdo (techo) y otra por su vértice inferior derecho (piso). Los demás caracteres quedan comprendidos en esa banda (con desviaciones menores a 2px), que determina la posición vertical de cualquier caracter faltante.

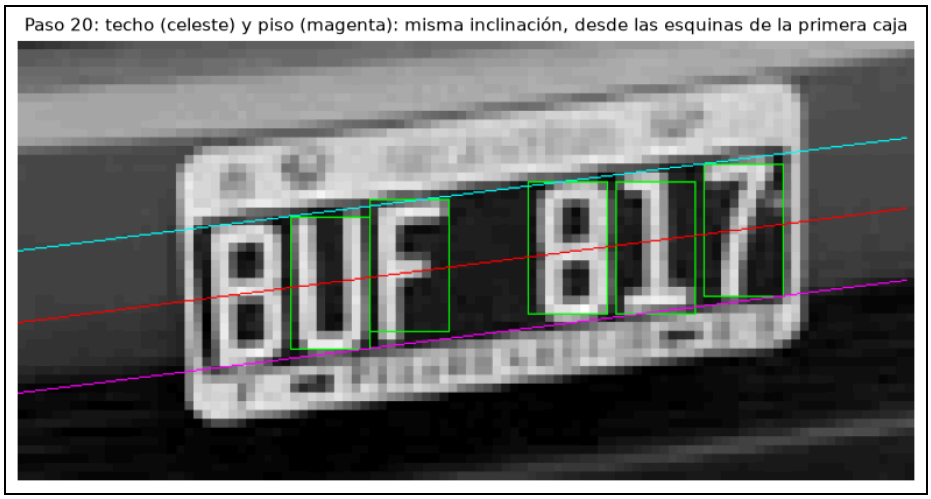

Figura 18 — Trazado de bandas de referencia.

3. **Eje de simetría vertical**. Se traza la vertical en el 50% del ancho de la ROI, que separa el grupo de letras del de dígitos. Cada lado debería contener 3 caracteres.

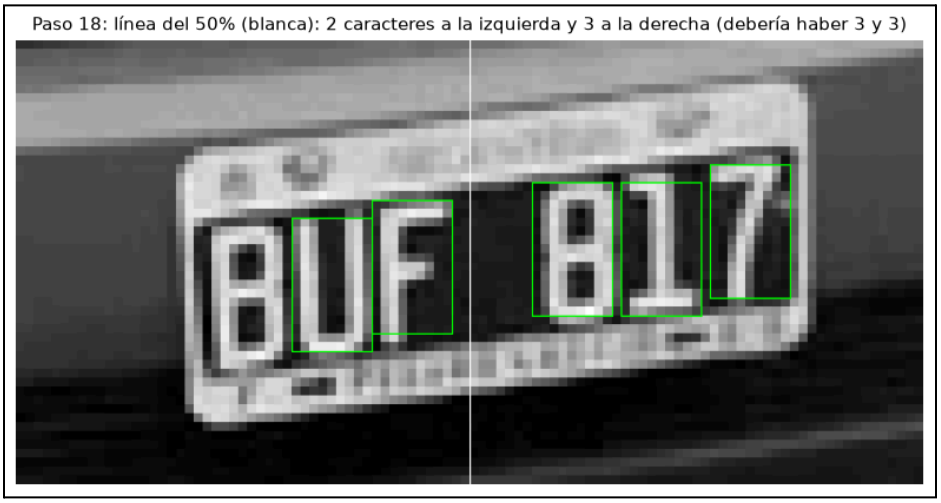

Figura 19 — Trazado del eje de simetría vertical.

4. **Extrapolación**. Si un lado tiene menos de 2 cajas, se determina dónde falta la tercera. Se miden las distancias hacia afuera desde el eje (así, ambos lados se tratan de forma simétrica). Sean A la caja más cercana al eje, B la más lejana, y “paso” la distancia entre sus centros:

Condición	**Posición inferida**
El espacio libre entre A y B es mayor que el ancho de una caja.	Entre ambas (punto medio)
A y B son contiguas y A esta a más de 1,5 pasos del eje (cabe otro caracter).	Hacia el eje (a un paso de A)
A y B son contiguas y A está próxima al eje	En el extremo (a un paso de B, hacia afuera)

Figura 20 — Caja inferida del simbolo no detectada.

Etapa 4 | Enderezado de la patente

Idea: una vez definidas las cajas de los 6 caracteres (detectadas y proyectadas), girar el recorte de modo que la fila de caracteres quede horizontal y recortar cada caracter de la imagen girada. La detección de las etapas anteriores se hizo sin rotar.

Para ello, llevamos a cabo el siguiente procedimiento:

1. **Ángulo de giro:** `θ = arctan(m)`, donde `m` es la pendiente de la recta que une los centros de la primera y la última caja detectada (la misma de la Figura 17).

Figura 21 — La línea que une los centroides determina el ángulo de rotación.

2. **Matriz de rotación:** transformación afín de matriz 2×3, con centro en el centro del recorte y escala 1 (`cv2.getRotationMatrix2D`). Con `α = cos θ` y `β = sin θ`: `M = [[α, β, (1−α)·cx − β·cy], [−β, α, β·cx + (1−α)·cy]]`.

3. **Giro del recorte:** `cv2.warpAffine` con interpolación bicúbica y tamaño de salida igual al de entrada. Las zonas que quedan vacías en las esquinas se completan por replicación del píxel de borde.

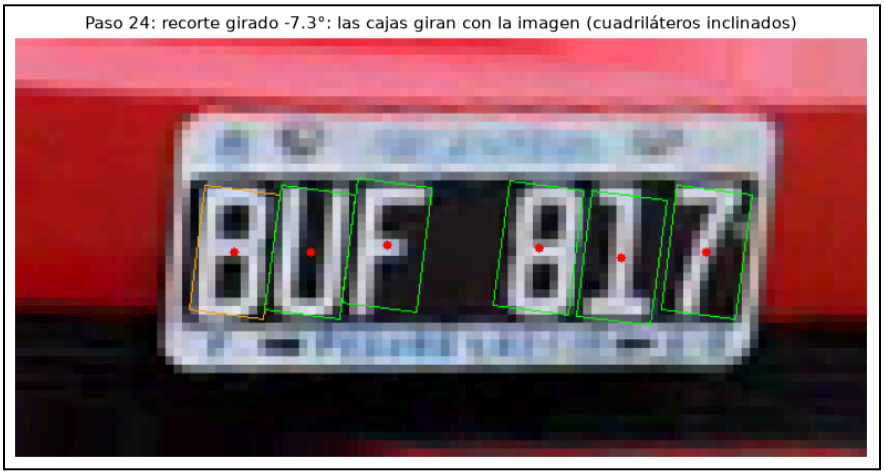

Figura 22 — Patente rotada.

4. **Transformación de las cajas:** se aplica la misma matriz `M` a los centros y a los vértices de cada caja, `x' = M·[x, y, 1]`^{T}. Un bounding box de lados paralelos a los ejes pasa a ser un cuadrilátero inclinado. Se reconstruye un rectángulo del mismo ancho y alto, centrado en el centro transformado.

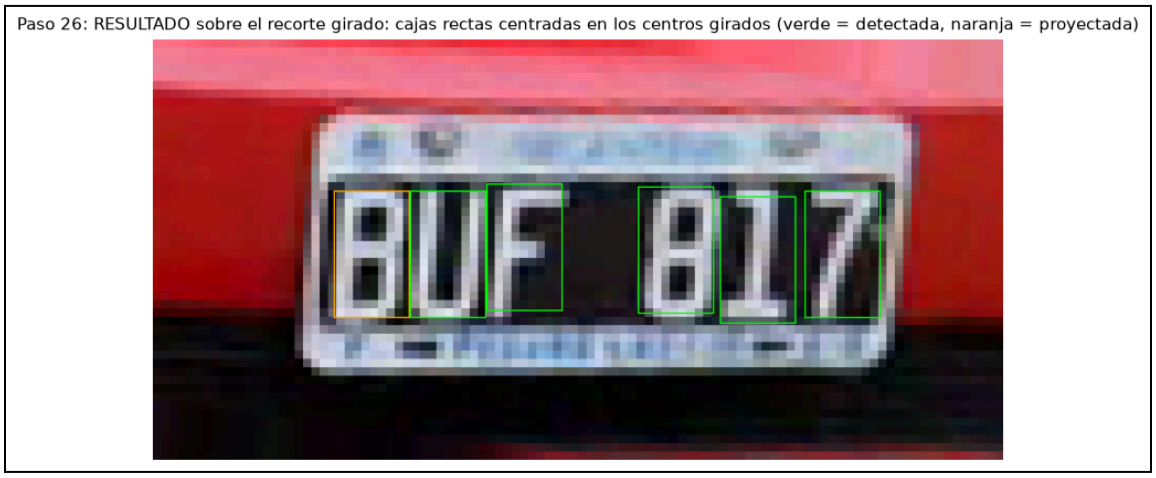

Figura 23 — Cajas ajustadas a la rotación.

5. **Verificación y extracción:** tras el giro, la pendiente entre el primer y el último centro es nula y el resto de los centros se desvía menos de 0,75 px de la fila (en las imágenes verificadas). Cada caracter se recorta de la imagen girada y se binariza con T = 127 sobre el gris estirado.

Resultados

Estado de la imagen 1 según la corrida completa.

Figura 24 — Dígitos detectados.

Caractéres obtenidos:

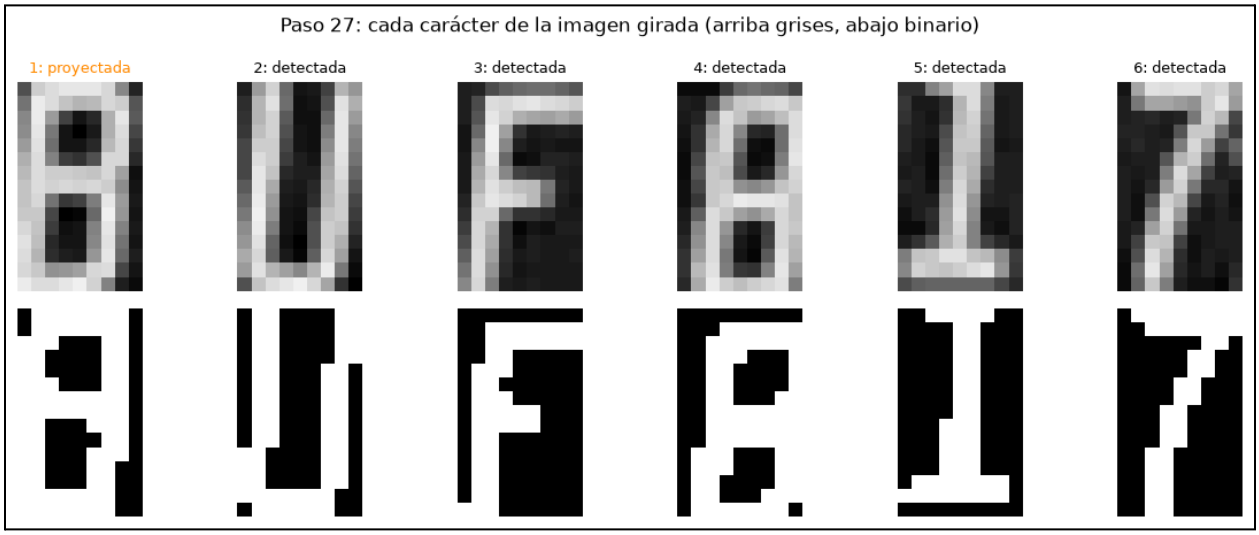

Figura 25 — Dígitos detectados.

Estado de cada imagen según la última corrida completa (capturas de las 12 imágenes).

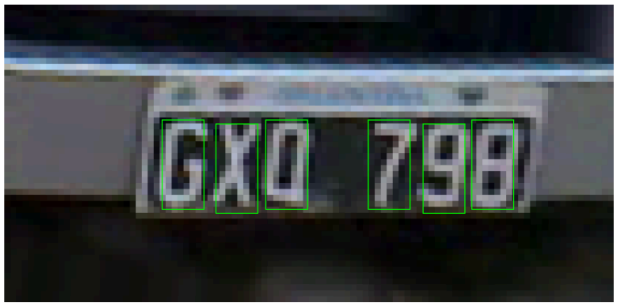

Figura 26 — Los digitos detectados en las 12 imágenes

Adicional

Para evaluar la robustez de la solución, procedimos a ponerla a prueba con fotos que hemos sacado en la calle de diferentes vehículos, con distinto ángulo, resolución y tipo de iluminación.

Resultados obtenidos:

Figura 27 — Se logró identificar correctamente todos los digitos de la patente.

Figura 28 — Se logró identificar correctamente todos los digitos de la patente.

Figura 29 — Solo se detectó el 4 y el 5, los demás dígitos no se encontraron.

Figura 30 — Se logró identificar correctamente todos los digitos de la patente.

Figura 31 — Se logró identificar correctamente todos los digitos de la patente.

Figura 32 — Ni siquiera se detectó correctamente la patente.

Conclusiones

Ambas soluciones lograron resolver el problema planteado con técnicas clásicas de procesamiento de imágenes: el Problema 1 permitió contar y clasificar las monedas y los dados, y el Problema 2 permitió localizar la patente y delimitar sus caracteres en las imágenes provistas. Como los parámetros se ajustaron y se evaluaron sobre el mismo conjunto de imágenes, sería conveniente probar ambas con un conjunto más amplio y variado para evaluar su robustez.

Además, se notó que ante grandes valores de iluminación la solución del Problema 2 tiende a dificultar la detección de las patentes: la patente deja de verse negra y pasa a reflejar el entorno, por lo que no se logra distinguir correctamente de lo que la rodea. Como siguiente paso, se podría intentar manejar estos casos, por ejemplo con una normalización local del contraste o un umbral adaptativo.